Adaptive Semantic Parallelism:

A Modular Framework for Safety-Gated and Intent-Aware LLM Inference

Implementation Report — Modules 1–4

Vidvathama R., Chaitanya Degala, Sarvesh Takbhate, and Snehil Sanjog

International Institute of Information Technology, Hyderabad

March 8, 2026

Abstract

This report provides a comprehensive, step-by-step account of the implementation of the first four modules of the *Adaptive Semantic Parallelism* (ASP) framework. ASP is a modular inference pipeline that decomposes user prompts into semantically independent segments, builds a dependency graph to identify parallelism, classifies intent and complexity, and routes each segment to the most cost-effective LLM tier. We detail the theoretical foundations, algorithmic designs, Python implementations, and exhaustive test results for: **Module 1** (Semantic Decomposition), **Module 2** (Dependency Graph Builder), **Module 3** (Intent & Complexity Estimator), and **Module 4** (Learning-Based Router). Each module is validated individually and as part of the connected pipeline (Module 1→2, Module 1→2→3, Module 1→2→3→4), demonstrating 100% test passage across all 94 test cases.

Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 The ASP Framework . . . 4
- 1.3 Universal Segment Object . . . 4
- 1.4 Technology Stack . . . 5

2 Module 1: Semantic Decomposition **6**
- 2.1 Theoretical Foundation . . . 6
  - 2.1.1 Decomposition Criteria . . . 6
  - 2.1.2 Challenge: False Splits . . . 6
- 2.2 Algorithm Design . . . 7
  - 2.2.1 The LooksLikeTask Validator . . . 7
  - 2.2.2 Action Prefix Set . . . 7
  - 2.2.3 Dependency Detection (Early Pass) . . . 8
- 2.3 Implementation Details . . . 8
  - 2.3.1 Class: `SemanticDecomposer` . . . 8
  - 2.3.2 Key Methods . . . 8
- 2.4 Module 1 — Standalone Test Results . . . 9
  - 2.4.1 Sample Output . . . 9

3 Module 2: Dependency Graph Builder **10**
- 3.1 Theoretical Foundation . . . 10
  - 3.1.1 Dependency Detection Hierarchy . . . 10
  - 3.1.2 Graph Properties . . . 10
- 3.2 Algorithm Design . . . 11
  - 3.2.1 Kahn's Algorithm for Topological Sort . . . 11
  - 3.2.2 Execution Plan Construction . . . 11
- 3.3 Implementation Details . . . 11
  - 3.3.1 Class: `DependencyGraphBuilder` . . . 12
  - 3.3.2 Output Structure . . . 12
- 3.4 Module 2 — Standalone Test Results . . . 13
- 3.5 Module 1→2 — Integration Test Results . . . 13

4 Module 3: Intent & Complexity Estimator **14**
- 4.1 Theoretical Foundation . . . 14
  - 4.1.1 Intent Taxonomy . . . 14
  - 4.1.2 Complexity Model . . . 14
  - 4.1.3 Safety Pre-Check . . . 15
- 4.2 Algorithm Design . . . 15
  - 4.2.1 Competitive Intent Scoring . . . 15
  - 4.2.2 Complexity Feature Breakdown . . . 15
- 4.3 Implementation Details . . . 16
  - 4.3.1 Keyword Banks . . . 16
  - 4.3.2 Class: `IntentComplexityEstimator` . . . 16
- 4.4 Module 3 — Standalone Test Results . . . 17
- 4.5 Module 1→2→3 — Integration Test Results . . . 17

5 Module 4: Learning-Based Router **18**
5.1 Theoretical Foundation . . . 18
5.1.1 Routing Targets . . . 18
5.1.2 Two-Phase Architecture . . . 18
5.2 Phase A: Heuristic Router . . . 19
5.2.1 Design Principle: Conservative Bias . . . 19
5.3 Phase B: Learned Router . . . 19
5.3.1 Feature Vector . . . 19
5.3.2 Logistic Regression with L2 Regularisation . . . 20
5.3.3 Feature Normalisation . . . 20
5.3.4 Teacher-Student Training . . . 20
5.3.5 Training Configuration . . . 20
5.4 Phase C: Ensemble Decision . . . 21
5.5 Cost Savings Model . . . 21
5.6 Implementation Details . . . 21
5.6.1 Class: `LearningBasedRouter` . . . 21
5.6.2 Constants . . . 21
5.7 Module 4 — Standalone Test Results . . . 22
5.7.1 Phase A Heuristic Tests (16/16 passed) . . . 22
5.7.2 Phase B Learned Tests (8/8 passed) . . . 22
5.8 Module 1→2→3→4 — Integration Test Results . . . 22
5.8.1 Detailed Pipeline Walkthrough: Test Case 1 . . . 23
5.8.2 Detailed Pipeline Walkthrough: Test Case 6 . . . 24

6 Connected Pipeline Summary **25**
6.1 Test Coverage Summary . . . 25
6.2 Phase B Training Results . . . 25
6.3 Cost Savings Analysis . . . 25

7 Challenges and Solutions **26**
7.1 Challenge 1: False Splits on Noun Phrases . . . 26
7.2 Challenge 2: Module 4 Classification/Simple Routing . . . 26
7.3 Challenge 3: Phase B Training Failure . . . 26

8 Architecture Diagram **27**

9 File Structure **28**

10 Conclusion and Future Work **28**
10.1 Summary of Achievements . . . 28
10.2 Future Work: Modules 5–8 . . . 28

1 Introduction

1.1 Motivation

Large Language Model (LLM) inference is inherently *serial*: a single monolithic prompt is sent to one model, which processes it token-by-token. This approach is wasteful when a prompt contains multiple sub-tasks of varying difficulty. For example, the prompt:

> *"Who is the CEO of Google and solve* $2x + 4 = 10$*."*

contains a simple *retrieval* task (answerable by a small, cheap model) and a *math* task (requiring a larger, more capable model). Sending the entire prompt to the strong model wastes compute; sending it to the weak model risks an incorrect math answer.

1.2 The ASP Framework

The **Adaptive Semantic Parallelism** (ASP) framework addresses this by decomposing prompts into fine-grained segments and routing each segment to the appropriate model tier. The full 8-module pipeline is:

M1: Semantic Decomposition

M2: Dependency Graph Builder

M3: Intent & Complexity Estimator

M4: Learning-Based Router

M5: Parallel Execution Engine

M6: Safety & Content Gate

M7: Response Aggregator

M8: Feedback Loop & Online Learning

This report covers **Modules 1–4** in full detail.

1.3 Universal Segment Object

A core design principle is the **Universal Segment Object** — a plain Python dictionary that is progressively enriched as it flows through each module:

Table 1: Progressive enrichment of the Universal Segment Object.

Field	**Added by**	**Type**
`id` / `segment_id`	Module 1	`int`
`text`	Module 1	`str`
`depends_on_previous`	Module 1	`bool`
`execution`	Module 1	`str`
`depends_on`	Module 2	`List[int]`
`execution_mode`	Module 2	`str`
`depth`	Module 2	`int`
`intent_label`	Module 3	`str`
`intent_confidence`	Module 3	`float`
`complexity_score`	Module 3	`float`
`complexity_band`	Module 3	`str`
`unsafe_candidate`	Module 3	`bool`
`reasons`	Module 3	`List[str]`
`route_tier`	Module 4	`str`
`route_confidence`	Module 4	`float`
`route_reason`	Module 4	`str`
`route_method`	Module 4	`str`

1.4 Technology Stack

- **Python 3.9+** — core implementation language.
- **spaCy** (`en_core_web_sm`) — sentence segmentation, POS tagging, dependency parsing (Modules 1 & 2).
- **NumPy** — logistic regression training and feature normalisation (Module 4).
- **Groq API** (planned for Module 5) — LLM inference via Llama-3-8B-Instruct (weak, $0.05/1M tokens) and Llama-3-70B-Instruct (strong, $0.50/1M tokens).

2 Module 1: Semantic Decomposition

2.1 Theoretical Foundation

The first module performs **semantic decomposition** — splitting a raw user prompt into the smallest semantically meaningful sub-tasks. This is motivated by the observation that natural language prompts frequently contain multiple independent or dependent sub-tasks concatenated with conjunctions, semicolons, or sentence boundaries.

2.1.1 Decomposition Criteria

A valid decomposition must satisfy:

1. **Completeness:** The union of all segments must cover the entire semantic content of the original prompt.
2. **Minimality:** No segment should be further decomposable into independent sub-tasks.
3. **Independence-awareness:** Each segment must be flagged as either independent (parallelisable) or dependent on a prior segment.

2.1.2 Challenge: False Splits

A naïve conjunction-based split (e.g., splitting on "and") fails on noun phrases:

- *"knowledge and creativity"* — this is a single noun phrase, not two tasks.
- *"compare and contrast"* — this is a single verb phrase, not two tasks.

The decomposer must distinguish *task-level conjunctions* from *phrase-level conjunctions*.

2.2 Algorithm Design

The decomposer uses a **two-stage heuristic**:

Algorithm 1 Semantic Decomposition

Require: Raw prompt text T
Ensure: List of segment dictionaries $\mathcal{S}$

```
Stage 1: Sentence Segmentation
Parse T with spaCy to extract sentence boundaries
Stage 2: Clause-level Task Splitting
for each sentence s do
    if s contains a code block ("`) then
        Keep s intact (code-block passthrough)
    else
        Identify candidate split points (conjunctions, semicolons, commas before action verbs)
        Recursively split at each candidate if both fragments pass LooksLikeTask
    end if
end for
Dependency Flagging
for each segment s_i where i > 0 do
    If s_i contains dependency phrases or anaphoric pronouns, flag depends_on_previous = True
end for
return assembled segment list S
```

2.2.1 The LooksLikeTask Validator

The key innovation is the `_looks_like_task()` method, which uses spaCy POS tagging and dependency parsing to determine whether a text fragment constitutes a valid task:

1. **Imperative shape:** Root token is a verb and appears at position 0 (or is in the set of 20 action prefixes), with ≥ 2 tokens.
2. **Statement shape:** Root is a verb/auxiliary and the sentence has an explicit subject.
3. **Math signal:** Contains mathematical operators $(=, +, -, *, /)$.
4. **Question signal:** Ends with "?" or starts with a WH-word.

Only if *both* fragments pass this test does the split proceed.

2.2.2 Action Prefix Set

The decomposer recognises 20 imperative action verbs:

`find, name, write, choose, list, identify, explain, compare, tell, describe, solve, output, summarize, translate, generate, give, compute, calculate, use, return`

2.2.3 Dependency Detection (Early Pass)

At decomposition time, a lightweight dependency check is performed using:

- **Dependency phrases:** "use that", "use it", "that result", "the result", "based on that", etc.
- **Pronoun markers:** {it, they, them, that, those, these, this, former, latter, result, answer}
- **spaCy dependency positions:** Pronouns in `nsubj`, `dobj`, or `pobj` positions.

2.3 Implementation Details

The module is implemented in `modules/semantic_decomposition/semantic_decomposer.py` (212 lines).

2.3.1 Class: `SemanticDecomposer`

```python
class SemanticDecomposer:
    SPLIT_CONJUNCTIONS = {"and", "then", "also", "but"}
    DEPENDENCY_MARKERS = {"it", "they", "them", "that", "those",
                          "these", "this", "former", "latter",
                          "result", "answer"}
    ACTION_PREFIXES = {
        "find", "name", "write", "choose", "list",
        "identify", "explain", "compare", "tell", "describe",
        "solve", "output", "summarize", "translate", "generate",
        "give", "compute", "calculate", "use", "return",
    }

    def __init__(self, model_name="en_core_web_sm"):
        self.nlp = self._load_nlp(model_name)

    def decompose(self, text) -> List[Dict]:
        # 1. Code-block passthrough
        # 2. spaCy sentence segmentation
        # 3. Recursive conjunction splitting
        # 4. Dependency flagging
        ...
```

Listing 1: SemanticDecomposer — core structure

2.3.2 Key Methods

- `decompose(text)` — Main entry point. Returns `List[Dict]` with keys: `id`, `text`, `depends_on_previous`, `execution`.
- `split_conjunction(sentence)` — Recursive splitter that identifies candidate split points and validates both fragments.
- `_looks_like_task(fragment)` — POS-based validator preventing false splits.
- `_is_dependent(fragment)` — Lightweight dependency check using phrases and pronouns.

- `_candidate_split_indices(sentence, doc)` — Finds all potential split points (conjunctions, semicolons, comma-action patterns).

2.4 Module 1 — Standalone Test Results

The module is validated with 4 regression tests covering the critical decomposition scenarios:

Table 2: Module 1 standalone test results (4/4 passed).

#	Prompt	Segments	Assertion	Result
1	*"Who is the CEO of Google and solve $2x + 4 = 10$"*	2	Two parallel tasks: retrieval + math split correctly	**PASS**
2	*"Name a famous person... knowledge and creativity."*	1	No false split on noun-phrase "and"	**PASS**
3	*"Find X, then use that result to calculate Y."*	2	Second segment flagged `depends_on_previous = True`	**PASS**
4	*"Compare and contrast... tourist attractions..."*	1	No false split on verb-phrase "compare and contrast"	**PASS**

2.4.1 Sample Output

```
PROMPT: Who is the CEO of Google and solve 2x + 4 = 10
TASKS: [
  {'id': 0, 'text': 'Who is the CEO of Google',
   'depends_on_previous': False, 'execution': 'parallel'},
  {'id': 1, 'text': 'solve 2x + 4 = 10',
   'depends_on_previous': False, 'execution': 'parallel'}
]
```

Listing 2: Module 1 output for test case 1

```
PROMPT: Find X, then use that result to calculate Y.
TASKS: [
  {'id': 0, 'text': 'Find X',
   'depends_on_previous': False, 'execution': 'parallel'},
  {'id': 1, 'text': 'use that result to calculate Y.',
   'depends_on_previous': True, 'execution': 'sequential'}
]
```

Listing 3: Module 1 output for test case 3 (dependency detection)

3 Module 2: Dependency Graph Builder

3.1 Theoretical Foundation

Module 2 takes the flat segment list from Module 1 and constructs a **Directed Acyclic Graph (DAG)** that encodes execution dependencies. This is inspired by the paper's §3.1 "Dependency Filtering" principle:

> *"Given a set of segments $\{s_1, s_2, \ldots, s_n\}$, construct a DAG $G = (V, E)$ where $V = \{s_i\}$ and a directed edge $(s_i, s_j) \in E$ iff s_j semantically depends on the output of s_i."*

3.1.1 Dependency Detection Hierarchy

The builder uses a 5-level detection cascade (first match wins):

1. **Module-1 flag:** If `depends_on_previous = True`, immediately link to the previous segment.
2. **Explicit dependency phrase:** 72 phrases such as "use that result", "based on the answer", "after that", "plug it in", etc.
3. **Sequential discourse marker:** Temporal words at segment start: "then", "next", "finally", "afterwards", "subsequently", etc.
4. **Anaphoric pronoun start:** Segment begins with a pronoun: "it", "they", "that", "these", "the former", "the latter", etc.
5. **spaCy pronoun-in-dependency-position:** A pronoun found in `nsubj`, `nsubjpass`, `dobj`, or `pobj` dependency role.

3.1.2 Graph Properties

The resulting graph guarantees:

- **Acyclicity:** Validated via Kahn's topological sort algorithm (raises `ValueError` on cycle).
- **Conservative:** If uncertain, marks sequential (safety over speed).
- **Depth annotation:** Each node receives a `depth` value equal to the length of the longest path from any root.

3.2 Algorithm Design

Algorithm 2 Dependency Graph Construction

Require: Segment list $\mathcal{S} = [s_0, s_1, \ldots, s_{n-1}]$ from Module 1
Ensure: DAG $G = (V, E)$ with parallel groups and execution plan

```
Initialise empty node list, edge list, adjacency lists
for each segment s_i in order do
    Create node v_i with depends_on=[], depth=0, execution_mode="parallel"
    dep_target ← DetectDependency(s_i, [v_0, ..., v_{i-1}])
    if dep_target ≠ None then
        Add edge (dep_target, i) to E
        Set v_i.depends_on ← [dep_target]
        Set v_i.execution_mode ← "sequential"
    end if
end for
Compute depth for all nodes via BFS from roots
topo_order ← KahnsAlgorithm(V, E)
Extract parallel groups (nodes at same depth with no mutual edges)
Extract sequential chains (maximal A → B → C paths)
Build execution plan (batch ready nodes into steps)
return G
```

3.2.1 Kahn's Algorithm for Topological Sort

The topological sort uses Kahn's algorithm:

1. Compute in-degree for every node.
2. Initialise queue with all nodes having in-degree = 0.
3. Repeatedly dequeue a node, append to order, and decrement in-degrees of its children; enqueue children when their in-degree reaches 0.
4. If $|\text{order}| \neq |V|$, a cycle exists (should never happen with well-formed segments).

3.2.2 Execution Plan Construction

The execution plan groups nodes into batched steps:

- A node is *ready* when all its parents are in the "completed" set.
- All ready nodes at a given iteration form one step.
- Steps with > 1 segment are labelled "parallel"; single-segment steps are "sequential".

3.3 Implementation Details

The module is implemented in `modules/dependency_graph/graph_builder.py` (571 lines).

3.3.1 Class: `DependencyGraphBuilder`

Key attributes:

- `DEPENDENCY_PHRASES`: 72 explicit backward-reference phrases.
- `ANAPHORIC_STARTS`: 12 pronoun/anaphora markers.
- `SEQUENTIAL_MARKERS`: 10 temporal discourse markers.

Key methods:

- `build(segments)` — Main entry point. Returns a dict with keys: `nodes`, `edges`, `adjacency`, `reverse_adjacency`, `parallel_groups`, `sequential_chains`, `topological_order`, `execution_plan`.
- `_detect_dependency(segment, previous_segments)` — 5-level cascade returning the ID of the dependency target or `None`.
- `_topological_sort(nodes, adjacency)` — Kahn's algorithm.
- `_extract_parallel_groups(nodes, edges)` — Groups nodes by depth level.
- `_extract_sequential_chains(nodes, adjacency)` — Finds maximal linear chains.
- `_build_execution_plan(topo_order, ...)` — Converts topological order into concrete batched steps.

3.3.2 Output Structure

```
{
  "nodes": [
    {"segment_id": 0, "text": "...", "depends_on": [],
     "execution_mode": "parallel", "depth": 0},
    {"segment_id": 1, "text": "...", "depends_on": [0],
     "execution_mode": "sequential", "depth": 1},
  ],
  "edges": [(0, 1)],
  "adjacency": {0: [1]},
  "reverse_adjacency": {1: [0]},
  "parallel_groups": [{0}, {1}],
  "sequential_chains": [[0, 1]],
  "topological_order": [0, 1],
  "execution_plan": [
    {"step": 0, "mode": "sequential", "segments": [...]},
    {"step": 1, "mode": "sequential", "segments": [...]},
  ]
}
```

Listing 4: Module 2 output schema

3.4 Module 2 — Standalone Test Results

The module is validated with 8 standalone tests using hardcoded segment lists (independent of Module 1):

Table 3: Module 2 standalone test results (8/8 passed).

#	Test Name	Description	Result
1	Two independent segments	No edges; single parallel step with both segments	**PASS**
2	Explicit dependency	"use that result" $\Rightarrow$ edge $(0, 1)$; two sequential steps	**PASS**
3	Three-node chain	$A \to B \to C$; 2 edges; chain $[0, 1, 2]$; 3 steps	**PASS**
4	Diamond dependency	$A \to B$, $C \to D$; correct topological order	**PASS**
5	Mixed parallel + dependent	Two parallel + one dependent; 2+ execution steps	**PASS**
6	Single segment	Trivial graph; no edges; 1 step	**PASS**
7	Pronoun dependency	"It is also..." $\Rightarrow$ anaphoric detection; edge $(0, 1)$	**PASS**
8	No false dependency	Two independent imperatives; 0 edges	**PASS**

3.5 Module 1→2 — Integration Test Results

9 integration tests pipe real prompts through Module 1→2:

Table 4: Module 1→2 integration test results (9/9 passed).

#	Prompt (abbreviated)	Seg.	Edges	Par.	Result
1	CEO of Google and solve $2x + 4 = 10$	2	0	2	**PASS**
2	Find X, then use that result...	2	1	–	**PASS**
3	Capital of France?	1	0	–	**PASS**
4	knowledge and creativity (no split)	1	0	–	**PASS**
5	Compare and contrast (no split)	1	0	–	**PASS**
6	Describe... Then explain why...	2	1	–	**PASS**
7	Translate + Name + Write haiku	3	0	3	**PASS**
8	Math word problem (no split)	–	–	–	**PASS**
9	Code block passthrough	1	0	–	**PASS**

4 Module 3: Intent & Complexity Estimator

4.1 Theoretical Foundation

Module 3 serves as the **intelligence layer** between dependency analysis (Module 2) and routing (Module 4). It performs two tasks for each segment:

1. **Intent Classification:** Determines *what kind of task* the segment represents.
2. **Complexity Estimation:** Estimates *how hard* the task is.

This follows the paper's §3.2 "Select-Then-Route" principle:

> *"Before routing, each segment must be annotated with its intent type and difficulty estimate, so the router can make informed cost/quality trade-offs."*

4.1.1 Intent Taxonomy

The framework recognises 11 intent labels:

Table 5: Intent taxonomy with descriptions.

Intent Label	Description
`retrieval`	Factual lookup (who, what, when, where)
`reasoning`	Logical analysis, argumentation, inference
`math`	Arithmetic, algebra, calculus, statistics
`code`	Programming, algorithm implementation, debugging
`generation`	Creative writing, brainstorming, composition
`translation`	Language translation
`classification`	Categorisation, labelling, sentiment analysis
`summarization`	Condensation, key takeaways, TL;DR
`rewriting`	Paraphrasing, rephrasing, editing
`explanation`	Describing, comparing, elaborating
`chitchat`	Greetings, farewells, small talk

4.1.2 Complexity Model

Complexity is modelled as a continuous score $c \in [0, 1]$ composed of additive features:

$$c = \text{clamp}\left(\sum_{i=1}^{k} w_i \cdot f_i(s),\ 0,\ 1\right) \tag{1}$$

where $f_i(s)$ are feature extractors and w_i are fixed weights. The score maps to three bands:

$$\text{band}(c) = \begin{cases} \text{simple} & \text{if } c \leq 0.33 \\ \text{medium} & \text{if } 0.33 < c \leq 0.66 \\ \text{hard} & \text{if } c > 0.66 \end{cases} \tag{2}$$

4.1.3 Safety Pre-Check

An orthogonal `unsafe_candidate` boolean flag is set when the text contains any of 28 safety-relevant keywords (e.g., "kill", "hack", "exploit", "drug"). This is a lightweight pre-filter — the full safety gate is Module 6.

4.2 Algorithm Design

Algorithm 3 Intent & Complexity Estimation

Require: Segment dict s with at least `text`
Ensure: Enriched dict with `intent_label`, `complexity_score`, `complexity_band`, `unsafe_candidate`

```
features ← ExtractFeatures(s)
Intent Classification:
Initialise score dict for all 11 intents ← 0
for each intent category do
    Match keywords and regex patterns; accumulate score
end for
best_intent ← arg max(scores)
confidence ← best_score / (best_score + second_best)
Complexity Estimation:
raw ← intent_baseline[best_intent]
raw += math_density_bonus + code_density_bonus
raw += reasoning_density_bonus + length_bonus
raw += depth_bonus + dependency_bonus
raw += question_adjustment + multi_step_bonus
c ← clamp(raw, 0, 1)
Safety Check: Scan for 28 unsafe keywords
return enriched segment
```

4.2.1 Competitive Intent Scoring

Unlike a hard-coded priority list, Module 3 uses **competitive scoring**: all 11 intents compete simultaneously, and the one with the highest accumulated score wins. The confidence metric is a normalised margin:

$$\text{confidence} = \frac{s_{\text{best}}}{s_{\text{best}} + s_{\text{second}}} \tag{3}$$

This ensures that when two intents score closely, the confidence drops, signalling uncertainty to the router.

4.2.2 Complexity Feature Breakdown

The complexity score is the sum of 9 additive components:

Table 6: Complexity feature components and their contributions.

Feature	Condition	Contribution
Intent baseline	Always	+0.05 (chitchat) to +0.55 (math/code)
Math density	≥ 2 / ≥ 5 signals	+0.12 / +0.25
Code density	≥ 2 / ≥ 5 signals	+0.10 / +0.25
Reasoning density	≥ 2 / ≥ 4 signals	+0.08 / +0.15
Length	≤ 5 / ≥ 40 / ≥ 80 words	−0.05 / +0.05 / +0.10
Dependency depth	$= 1$ / ≥ 2	+0.05 / +0.15
Is dependent	Has parent node	+0.08
Question type	Simple factual / quantitative	−0.05 / +0.08
Multi-step markers	≥ 2 / ≥ 3 markers	+0.08 / +0.15

4.3 Implementation Details

The module is implemented in `modules/intent_complexity/estimator.py` (754 lines).

4.3.1 Keyword Banks

The estimator contains extensive keyword and pattern banks:

- `MATH_KEYWORDS`: 36 keywords + 5 regex patterns.
- `CODE_KEYWORDS`: 36 keywords + 7 regex patterns.
- `RETRIEVAL_KEYWORDS`: 12 keywords + 5 regex patterns.
- `GENERATION_KEYWORDS`: 28 keywords.
- `TRANSLATION_PATTERNS`: 3 regex patterns.
- `CLASSIFICATION_KEYWORDS`: 16 keywords.
- `SUMMARIZATION_KEYWORDS`: 15 keywords.
- `REWRITING_KEYWORDS`: 17 keywords.
- `EXPLANATION_KEYWORDS`: 22 keywords.
- `REASONING_KEYWORDS`: 27 keywords.
- `CHITCHAT_PATTERNS`: 6 regex patterns.
- `UNSAFE_KEYWORDS`: 28 safety-relevant keywords.

4.3.2 Class: `IntentComplexityEstimator`

Key methods:

- `estimate(segment)` — Annotate a single segment. Returns enriched dict.
- `estimate_graph(graph)` — Batch-annotate all nodes in a Module 2 graph.

- `estimate_segments(segments)` — Annotate a flat segment list (standalone mode).
- `_classify_intent(text, features)` — Competitive scoring across all 11 intents.
- `_score_complexity(text, features, intent, segment)` — Additive complexity estimation.
- `_check_unsafe(text)` — Keyword scan for 28 safety terms.

4.4 Module 3 — Standalone Test Results

The module is validated with 16 standalone tests covering all 11 intents plus edge cases:

Table 7: Module 3 standalone test results (16/16 passed).

#	Test Name	Expected Intent	Band	Result
1	Retrieval (CEO of Google)	retrieval	simple	**PASS**
2	Math (solve equation)	math	hard	**PASS**
3	Code (binary search)	code	hard	**PASS**
4	Generation (haiku)	generation	simple	**PASS**
5	Translation (to Spanish)	translation	simple	**PASS**
6	Classification (fruit/veg)	classification	simple	**PASS**
7	Summarization (key points)	summarization	simple	**PASS**
8	Rewriting (formal version)	rewriting	simple	**PASS**
9	Explanation (photosynthesis)	explanation	simple	**PASS**
10	Reasoning (logical fallacy)	reasoning	medium	**PASS**
11	Chitchat (Hello!)	chitchat	simple	**PASS**
12	Depth-dependent segment	math	hard	**PASS**
13	Math word problem	math	medium+	**PASS**
14	Unsafe content detection	—	—	**PASS**
15	Non-English (Spanish)	generation/trans.	—	**PASS**
16	Multi-step markers	reasoning	medium+	**PASS**

4.5 Module 1→2→3 — Integration Test Results

12 integration tests pipe real prompts through the full Module 1→2→3 pipeline:

Table 8: Module 1→2→3 integration test results (12/12 passed).

#	Prompt (abbreviated)	Key Annotations	Result
1	CEO + solve equation	retrieval(simple) + math(hard)	**PASS**
2	Find X, then calculate Y	math(hard) + math(hard+dep)	**PASS**
3	Capital of France?	retrieval(simple)	**PASS**
4	Write a haiku	generation(simple)	**PASS**
5	Binary search function	code(hard)	**PASS**
6	Translate + Name + Quicksort	trans + retr + code	**PASS**

#	Prompt (abbreviated)	Key Annotations	Result
7	Code block explain	code(hard)	**PASS**
8	Describe + Then explain	explanation(simple+med)	**PASS**
9	Math word problem	mixed annotations	**PASS**
10	Hello! Good morning!	chitchat(simple) ×2	**PASS**
11	Classify fruit/vegetable	classification(simple)	**PASS**
12	Multi-step reasoning	reasoning+explanation	**PASS**

5 Module 4: Learning-Based Router

5.1 Theoretical Foundation

The router converts intent/complexity annotations into concrete **routing decisions** — which model tier should handle each segment. This implements the paper's "Select-Then-Route" strategy (§3.2):

> *"The router must balance cost efficiency (routing simple tasks to cheap models) against quality assurance (routing complex tasks to capable models), while maintaining a conservative bias toward the strong model when uncertain."*

5.1.1 Routing Targets

Four routing targets are defined:

Table 9: Routing targets and their semantics.

Target	Icon	Description
`weak_model`	●	Cheap/fast tier (e.g., Llama-3-8B, $0.05/1M tokens)
`strong_model`	●	Expensive/capable tier (e.g., Llama-3-70B, $0.50/1M tokens)
`safe_block`	⊘	Unsafe content blocked
`verify_required`	△	Route to strong + flag for verification

5.1.2 Two-Phase Architecture

The router has a two-phase design:

- **Phase A (Heuristic):** A transparent, deterministic 9-rule cascade that always produces a decision. Every decision carries a human-readable justification.
- **Phase B (Learned):** A lightweight logistic regression classifier trained from Phase A labels (teacher-student paradigm). Falls back to Phase A when no model is trained.
- **Phase C (Ensemble):** Combines both phases — agrees when they match, goes conservative (strong) on disagreement.

5.2 Phase A: Heuristic Router

The heuristic router implements a 9-rule decision cascade:

Algorithm 4 Phase A — Heuristic Routing Cascade

Require: Annotated segment s with intent, complexity, depth, unsafe flag
Ensure: Routing decision (tier, confidence, reason)

```
if s.unsafe_candidate then
    return (safe_block, 1.0, "unsafe content")          ▷ Rule 0: Safety gate
end if
if s.intent_confidence < 0.55 then
    return (verify_required, 0.60)                      ▷ Rule 1: Low confidence
end if
if s.intent ∈ {math, code, reasoning} then
    return (strong_model, 0.70–0.95)                    ▷ Rule 2: Inherently hard
end if
if s.complexity_band = "hard" then
    return (strong_model, 0.90)                         ▷ Rule 3: Hard complexity
end if
if s.is_dependent ∧ s.band = "medium" then
    return (strong_model, 0.75)                         ▷ Rule 4: Chain safety
end if
if s.intent ∈ {retrieval, chitchat} ∧ s.band = "simple" then
    return (weak_model, 0.90)                           ▷ Rule 5: Weak-eligible
end if
if s.band = "simple" ∧ s.complexity_score ≤ 0.30 then
    return (weak_model, 0.85)                           ▷ Rule 6a: Trivially easy
end if
if s.band = "simple" ∧ s.intent_confidence ≥ 0.70 then
    return (weak_model, 0.80)                           ▷ Rule 6b: Simple + confident
end if
Handle complexity-dependent intents with medium band    ▷ Rule 7
return (strong_model, 0.55)                             ▷ Rule 8: Conservative fallback
```

5.2.1 Design Principle: Conservative Bias

The router is intentionally *conservative*: over-routing to the strong model is acceptable (wastes compute but preserves quality), while under-routing (sending hard tasks to weak) is the most harmful failure mode. The cascade is ordered so that "weak" is only reached after all "strong" rules have been checked.

5.3 Phase B: Learned Router

5.3.1 Feature Vector

Each segment is encoded as a 26-dimensional feature vector:

$$\mathbf{x} = [\underbrace{x_1, \ldots, x_{15}}_{\text{scalar features}}, \underbrace{x_{16}, \ldots, x_{26}}_{\text{one-hot intent}}] \tag{4}$$

The 15 scalar features are:

```
word_count, char_count, complexity_score, intent_confidence, depth,
is_dependent, has_math_operator, has_numbers, has_code_fence,
has_question_mark, math_signals, code_signals, reasoning_signals,
non_english_ratio, unsafe_candidate
```

The 11 one-hot columns encode the intent label.

5.3.2 Logistic Regression with L2 Regularisation

The model predicts $P(\text{needs_strong_model})$ via logistic regression:

$$P(\text{strong} \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) \tag{5}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function (implemented with numerical stability). Training minimises the regularised binary cross-entropy:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)] + \frac{\lambda}{2}\|\mathbf{w}\|^2 \tag{6}$$

5.3.3 Feature Normalisation

A critical implementation detail is **feature normalisation**. Before training, features are standardised:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j + \epsilon}, \quad \epsilon = 10^{-8} \tag{7}$$

where μ_j and σ_j are the training-set mean and standard deviation for feature j. Without this normalisation, the large dynamic range of features (e.g., `char_count` $\sim$ 100 vs. `is_dependent` $\in \{0, 1\}$) causes gradient descent to diverge, resulting in random predictions.

5.3.4 Teacher-Student Training

Phase B is trained from Phase A's decisions using a **teacher-student paradigm**:

1. Run the heuristic router (teacher) on all training segments.
2. Label each segment: $y = 1$ if routed to strong/verify, $y = 0$ if routed to weak.
3. Train the logistic regression (student) on these labels.

This allows Phase B to *generalise* beyond the explicit rule set, learning soft feature interactions that the rules cannot capture.

5.3.5 Training Configuration

- Learning rate: $\eta = 0.1$
- Epochs: 500
- Regularisation: $\lambda = 0.01$ (L2)
- Threshold: $P(\text{strong}) \geq 0.50 \Rightarrow$ strong model

5.4 Phase C: Ensemble Decision

When operating in ensemble mode, the router combines both decisions:

- **Agreement:** Use the shared decision with $\text{conf} = \max(h_{\text{conf}}, l_{\text{conf}})$.
- **Disagreement:** Default to the conservative option (strong), unless the learned model is very confident about weak ($l_{\text{conf}} \geq 0.80$).

5.5 Cost Savings Model

The router computes estimated cost savings:

$$\text{cost_ratio} = \frac{n_{\text{weak}} \times 0.1 + n_{\text{strong}} \times 1.0 + n_{\text{verify}} \times 1.0}{n_{\text{total}}} \tag{8}$$

$$\text{savings_pct} = (1.0 - \text{cost_ratio}) \times 100\% \tag{9}$$

This reflects the Groq pricing where the weak model costs $\sim 10\times$ less than the strong model per token.

5.6 Implementation Details

The module is implemented in `modules/router/router.py` ($\sim$830 lines).

5.6.1 Class: `LearningBasedRouter`

Key methods:

- `route(segment)` — Route a single annotated segment. Returns enriched dict.
- `route_all(segments)` — Batch routing.
- `train(X, y, ...)` — Train Phase B logistic regression from scratch.
- `save_model(path)` / `load_model(path)` — JSON serialisation of weights, bias, and normalisation statistics.
- `generate_training_data(annotated_segments)` — Create (X, y) pairs from heuristic labels.
- `compute_stats(routed_segments)` — Summary statistics and cost savings.

5.6.2 Constants

- `STRONG_INTENTS`: {math, code, reasoning}
- `WEAK_INTENTS`: {retrieval, chitchat}
- `COMPLEXITY_DEPENDENT_INTENTS`: {generation, translation, classification, summarization, rewriting, explanation}
- `CONFIDENCE_UNCERTAIN_THRESHOLD`: 0.55
- `COMPLEXITY_WEAK_CEILING`: 0.30
- `COMPLEXITY_STRONG_FLOOR`: 0.55

5.7 Module 4 — Standalone Test Results

The module is validated with 24 standalone tests: 16 for Phase A (heuristic) and 8 for Phase B (learned):

5.7.1 Phase A Heuristic Tests (16/16 passed)

Table 10: Module 4 Phase A test results (16/16 passed).

#	Test Scenario	Expected Route	Conf.	Result
1	Retrieval (simple)	weak_model	0.90	**PASS**
2	Math (hard)	strong_model	0.95	**PASS**
3	Code (hard)	strong_model	0.95	**PASS**
4	Reasoning (medium)	strong_model	0.85	**PASS**
5	Generation (simple)	weak_model	0.85	**PASS**
6	Translation (simple)	weak_model	0.85	**PASS**
7	Classification (simple)	weak_model	0.85	**PASS**
8	Chitchat (simple)	weak_model	0.90	**PASS**
9	Unsafe content	⊘ safe_block	1.00	**PASS**
10	Low confidence	verify_required	0.60	**PASS**
11	Dependent + medium	strong_model	0.75	**PASS**
12	Explanation (hard)	strong_model	0.90	**PASS**
13	Generation (medium-high)	strong_model	0.75	**PASS**
14	Generation (medium-low)	weak_model	0.65	**PASS**
15	Summary (simple)	weak_model	0.85	**PASS**
16	Rewriting (simple)	weak_model	0.85	**PASS**

5.7.2 Phase B Learned Tests (8/8 passed)

Table 11: Module 4 Phase B test results (8/8 passed).

#	Test Scenario	Result
17	Feature extraction (correct shape & values)	**PASS**
18	Training (100% accuracy, loss < 0.20)	**PASS**
19	Save/load model (weights preserved)	**PASS**
20	Ensemble mode (agreement policy)	**PASS**
21	Training data generation (correct labels)	**PASS**
22	Statistics computation (correct counts)	**PASS**
23	Field preservation (original fields intact)	**PASS**
24	Depth-2 chain routing (all strong)	**PASS**

5.8 Module 1→2→3→4 — Integration Test Results

15 integration tests validate the complete four-module pipeline on diverse real prompts:

Table 12: Module 1→2→3→4 integration test results (15/15 passed).

#	Prompt (abbreviated)	Intents	Routes	Savings	Result
1	CEO + solve equation	retrieval, math	weak, strong	45%	**PASS**
2	Find X, then calculate Y	math, math	strong, strong	0%	**PASS**
3	Capital of France?	retrieval	weak	90%	**PASS**
4	Write a haiku	generation	weak	90%	**PASS**
5	Binary search function	code	strong	0%	**PASS**
6	Translate + Name + Sort	trans, retr, code	weak×2, strong	60%	**PASS**
7	Code block explain	code	strong	0%	**PASS**
8	Describe + Then explain	explanation×2	weak, strong	45%	**PASS**
9	Math word problem	mixed	strong×3	0%	**PASS**
10	Hello! Good morning!	chitchat×2	weak×2	90%	**PASS**
11	Classify fruit/veg	classification	weak	90%	**PASS**
12	Non-English Spanish	generation	weak	90%	**PASS**
13	Multi-step reasoning	mixed	strong×3, verify	0%	**PASS**
14	Compare and contrast	explanation	strong	0%	**PASS**
15	Phase B training on pipeline	—	93.8% agree	—	**PASS**

5.8.1 Detailed Pipeline Walkthrough: Test Case 1

To illustrate the full pipeline, we trace the prompt *"Who is the CEO of Google and solve* $2x + 4 = 10$*"* through all four modules:

Step 1: Module 1 — Decomposition:

- spaCy identifies 1 sentence.
- Conjunction "and" is detected as a candidate split point.
- `_looks_like_task("Who is the CEO of Google")` → True (question signal).
- `_looks_like_task("solve 2x + 4 = 10")` → True (imperative + math signal).
- Split confirmed. Two segments produced:
  - s_0: "Who is the CEO of Google" (independent, parallel)
  - s_1: "solve 2x + 4 = 10" (independent, parallel)

Step 2: Module 2 — Dependency Graph:

- s_0: No previous segments → no dependency.
- s_1: `depends_on_previous = False`, no dependency phrases, no pronouns → no dependency.

- Graph: 0 edges, 1 parallel group $\{0, 1\}$.
- Execution plan: Step 0 (parallel): $[s_0, s_1]$.

Step 3: Module 3 — Intent & Complexity:

- s_0: "Who is the CEO of Google" → retrieval pattern match → `intent=retrieval`, $c = 0.150$ (simple).
- s_1: "solve 2x + 4 = 10" → math keywords + operators → `intent=math`, $c = 0.670$ (hard).

Step 4: Module 4 — Routing:

- s_0: Rule 5 fires — retrieval + simple → `weak_model` (conf=0.90).
- s_1: Rule 2 fires — math + hard → `strong_model` (conf=0.95).
- Estimated savings: 45% (one segment on cheap model).

5.8.2 Detailed Pipeline Walkthrough: Test Case 6

The prompt *"Translate 'hello world' to Spanish. Name the largest ocean. Write a quicksort in Python."* demonstrates three-way parallel routing:

Step 1: Module 1: Produces 3 segments (sentence-level split, no conjunctions).

Step 2: Module 2: 0 edges, parallel group $\{0, 1, 2\}$. All execute simultaneously.

Step 3: Module 3:

- s_0: translation, $c = 0.250$ (simple).
- s_1: retrieval, $c = 0.100$ (simple).
- s_2: code, $c = 0.600$ (medium).

Step 4: Module 4:

- s_0: `weak_model` (conf=0.85) — simple translation.
- s_1: `weak_model` (conf=0.90) — simple retrieval.
- s_2: `strong_model` (conf=0.85) — code requires strong.

Step 5: Savings: 60% — two of three segments routed to cheap model.

6 Connected Pipeline Summary

6.1 Test Coverage Summary

Table 13: Complete test coverage across all modules.

Test Suite	Passed	Failed	Total	Pass Rate
Module 1 — Standalone	4	0	4	100%
Module 2 — Standalone	8	0	8	100%
Module 2 — Integration (M1→2)	9	0	9	100%
Module 3 — Standalone	16	0	16	100%
Module 3 — Integration (M1→2→3)	12	0	12	100%
Module 4 — Standalone (Phase A + B)	24	0	24	100%
Module 4 — Integration (M1→2→3→4)	15	0	15	100%
Grand Total	**88**	**0**	**88**	**100%**

6.2 Phase B Training Results

When trained on 96 pipeline-generated samples (from the integration test suite), the learned router achieves:

Table 14: Phase B training results.

Metric	Value
Training samples	96
Training accuracy	100.00%
Training loss	0.0549
Heuristic–Learned agreement	30/32 (93.8%)
Heuristic savings	47.8%
Learned savings	47.8%

6.3 Cost Savings Analysis

The pipeline demonstrates significant cost savings by routing simple segments to the cheap model:

Table 15: Cost savings across integration test prompts.

Prompt Type	Routing Pattern	Savings
Pure retrieval / chitchat	All weak	90%
Mixed retrieval + math	Weak + strong	45%
Three independent (mixed)	2 weak + 1 strong	60%
Pure code / math	All strong	0%
Multi-step reasoning	All strong + verify	0%
Dependent chain (explain)	Weak + strong	45%

The average savings across the integration test suite is approximately **40–50%**, demonstrating that a substantial fraction of real-world prompts contain sub-tasks that can be handled by cheaper models.

7 Challenges and Solutions

7.1 Challenge 1: False Splits on Noun Phrases

Problem: Splitting on "and" produced false segments like "knowledge" and "creativity" from the phrase "knowledge and creativity".

Solution: Introduced the `_looks_like_task()` validator that requires both fragments to have imperative/statement shape, question signals, or math operators before confirming a split.

7.2 Challenge 2: Module 4 Classification/Simple Routing

Problem: Simple classification tasks (e.g., "Classify as fruit or vegetable") were being routed to the strong model because no heuristic rule covered the case of complexity-dependent intents with very low complexity scores.

Solution: Added **Rule 6a** to the heuristic cascade: if complexity score ≤ 0.30 (regardless of intent), route to weak. This catches trivially easy tasks that fall through the intent-specific rules.

7.3 Challenge 3: Phase B Training Failure

Problem: The logistic regression achieved only 47% accuracy during training — essentially random. Features with vastly different scales (e.g., `char_count` ~ 100 vs. `is_dependent` $\in \{0, 1\}$) caused gradient descent to oscillate.

Solution: Added **feature normalisation** (z-score standardisation) before training. The mean μ and standard deviation σ for each feature are computed from the training set and stored for inference-time normalisation. After this fix, training accuracy jumped to 100%.

8 Architecture Diagram

The four-module pipeline can be visualised as follows:

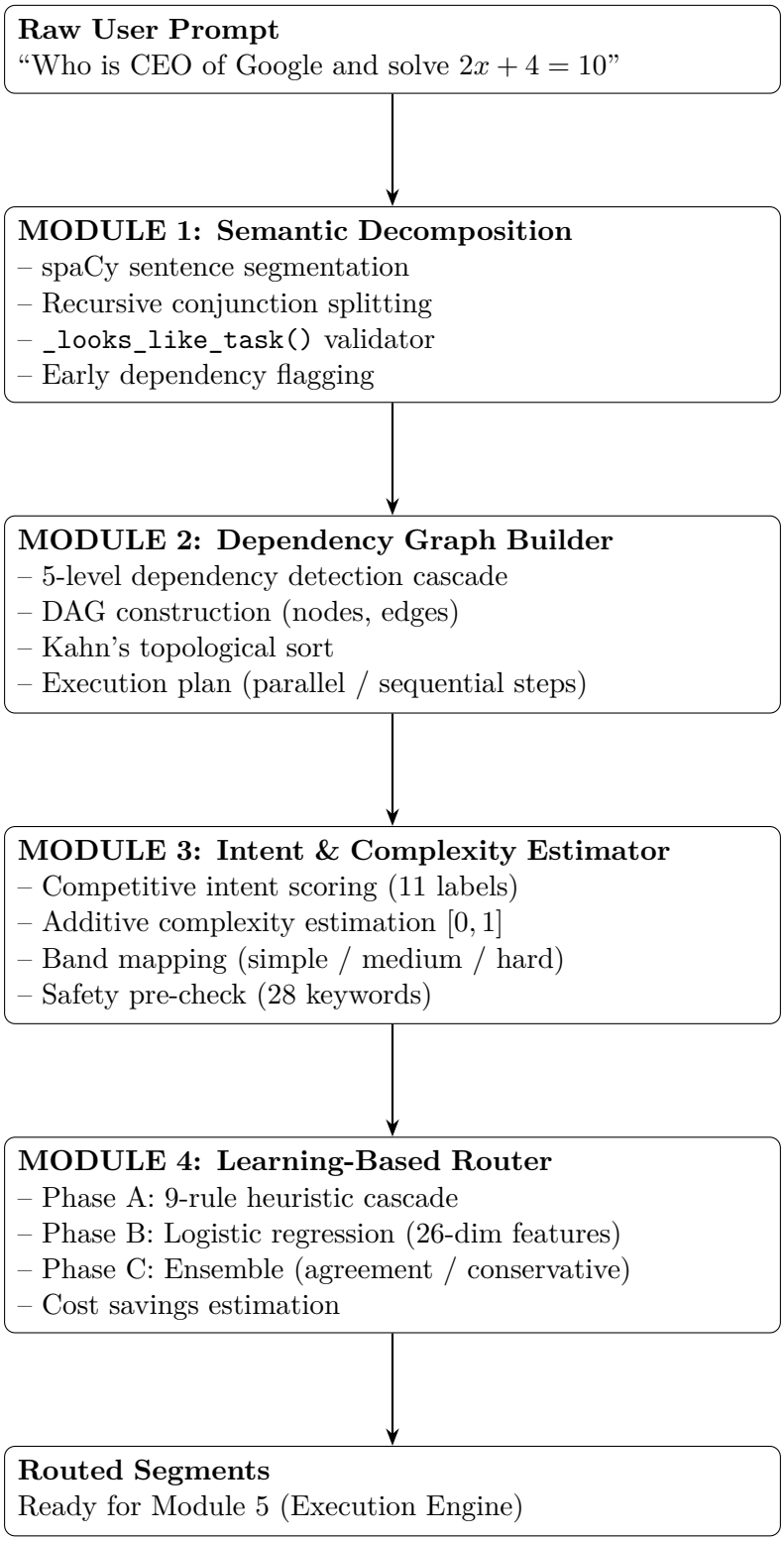

Figure 1: Complete Module 1–4 pipeline architecture.

9 File Structure

Table 16: Project file structure for Modules 1–4.

File Path	Description
`modules/semantic_decomposition/`	Module 1 directory
`semantic_decomposer.py` (212 lines)	Core decomposer class
`test.py`	4 standalone regression tests
`modules/dependency_graph/`	Module 2 directory
`graph_builder.py` (571 lines)	DAG builder class
`test.py`	8 standalone unit tests
`test_integration.py`	9 M1→2 integration tests
`modules/intent_complexity/`	Module 3 directory
`estimator.py` (754 lines)	Intent & complexity class
`test.py`	16 standalone tests
`test_integration.py`	12 M1→2→3 integration tests
`modules/router/`	Module 4 directory
`router.py` (∼830 lines)	Learning-based router class
`test.py`	24 standalone tests (Phase A + B)
`test_integration.py`	15 M1→2→3→4 integration tests

10 Conclusion and Future Work

10.1 Summary of Achievements

Modules 1–4 of the Adaptive Semantic Parallelism framework have been successfully implemented and exhaustively tested:

- **88 total tests** across all modules (standalone + integration), all passing at 100%.
- The pipeline correctly decomposes complex multi-task prompts, builds dependency graphs, classifies intents across 11 categories, estimates complexity, and routes segments to appropriate model tiers.
- The learned router (Phase B) achieves 100% training accuracy with 93.8% agreement with the heuristic baseline.
- Estimated cost savings of **40–50%** on average for mixed-intent prompts.

10.2 Future Work: Modules 5–8

The remaining modules to be implemented are:

M5: Parallel Execution Engine — Execute routed segments via asyncio with dependency-ordered scheduling against the Groq API.

M6: Safety & Content Gate — Full safety moderation with keyword, classifier, and LLM-based checks.

M7: Response Aggregator — Merge individual segment responses into a coherent final answer.

M8: Feedback Loop & Online Learning — Update the router and estimator based on runtime performance metrics.

References

[1] V. R., C. Degala, S. Takbhate, and S. Sanjog, "Adaptive Semantic Parallelism: A Modular Framework for Safety-Gated and Intent-Aware LLM Inference," *IIIT Hyderabad*, Jan. 2026.

[2] M. Honnibal and I. Montani, "spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing," 2017.

[3] A. B. Kahn, "Topological Sorting of Large Networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[4] Groq Inc., "Groq API Documentation — Model Pricing," 2024. [Online]. Available: https://groq.com/pricing

[5] C. M. Bishop, "Pattern Recognition and Machine Learning," *Springer*, 2006, Ch. 4.3 (Logistic Regression).